Simulation of Robotic Systems Lab 02

Lätitzia Brandt

May 16, 2025

Task 1

Start the program CoppeliaSim and load the scene `euler_angle.ttt`. You will see only one reference frame. Please note, that usage of euler angles in CoppeliaSim is based by the order X-Y-Z. Calculate the total rotation matrix:

$R(\alpha, \beta, \gamma) = R_x(\alpha) \cdot R_y(\beta) \cdot R_z(\gamma)$ on paper (two 3-by-3 matrix multiplications).

Additionally note, that quaternions are represented differently in CoppeliaSim compare to the usual representation q = (w,x,y,z) T.

You can use the file `P2_Task1.py` to find out.

Implement the functions within the Python file.

Prep

$$R_x(\alpha) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & cos\alpha & -sin\alpha \\ 0 & sin\alpha & cos\alpha \end{bmatrix}$$

$$R_y(\beta) = \begin{bmatrix} cos\beta & 0 & sin\beta \\ 0 & 1 & 0 \\ -sin\beta & 0 & cos\beta \end{bmatrix}$$

$$R_z(\gamma) = \begin{bmatrix} cos\gamma & -sin\gamma & 0 \\ sin\gamma & cos\gamma & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Step 1: $R(\alpha, \beta) = R_x(\alpha) \cdot R_y(\beta)$

$$R_y(\beta) = \begin{bmatrix} cos\beta & 0 & sin\beta \\ 0 & 1 & 0 \\ -sin\beta & 0 & cos\beta \end{bmatrix}$$

$$R_x(\alpha) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & cos\alpha & -sin\alpha \\ 0 & sin\alpha & cos\alpha \end{bmatrix} \quad \begin{bmatrix} c_\beta & 0 & s_\beta \\ s_\alpha \cdot s_\beta & c_\alpha & -s_\alpha \cdot c_\beta \\ -c_\alpha \cdot s_\beta & s_\alpha & c_\alpha \cdot c_\beta \end{bmatrix}$$

Step 2: $R(\alpha, \beta, \gamma) = R_{xy}(\alpha, \beta) \cdot R_z(\gamma)$

$$R_z(\gamma) = \begin{bmatrix} cos\gamma & -sin\gamma & 0 \\ sin\gamma & cos\gamma & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$R_{xy}(\alpha, \beta) = \begin{bmatrix} c_\beta & 0 & s_\beta \\ s_\alpha \cdot s_\beta & c_\alpha & -s_\alpha \cdot c_\beta \\ -c_\alpha \cdot s_\beta & s_\alpha & c_\alpha \cdot c_\beta \end{bmatrix} \begin{bmatrix} c_\beta \cdot c_\gamma & -c_\beta \cdot s_\gamma & s_\beta \\ s_\alpha \cdot s_\beta \cdot c_\gamma + c_\alpha \cdot s_\gamma & -s_\alpha \cdot s_\beta \cdot s_\gamma + c_\alpha \cdot c_\gamma & -s_\alpha \cdot c_\beta \\ -c_\alpha \cdot s_\beta \cdot c_\gamma + s_\alpha \cdot s_\gamma & c_\alpha \cdot s_\beta \cdot s_\gamma + s_\alpha \cdot c_\gamma & c_\alpha \cdot c_\beta \end{bmatrix}$$

Implementation 1)

```
def convert_quaternion_to_matrix(quat):
    # in CoppeliaSim: quaternion [qx qy qz qw]
    qx = quat[0]
    qy = quat[1]
    qz = quat[2]
    qw = quat[3]

    # First row of the rotation matrix
    r00 = 2 * (qw * qw + qx * qx) - 1
    r01 = 2 * (qx * qy - qw * qz)
    r02 = 2 * (qx * qz + qw * qy)

    # Second row of the rotation matrix
    r10 = 2 * (qx * qy + qw * qz)
    r11 = 2 * (qw * qw + qy * qy) - 1
    r12 = 2 * (qy * qz - qw * qx)

    # Third row of the rotation matrix
    r20 = 2 * (qx * qz - qw * qy)
    r21 = 2 * (qy * qz + qw * qx)
    r22 = 2 * (qw * qw + qz * qz) - 1

    # 3x3 rotation matrix
    rot_matrix = np.array([[r00, r01, r02],
    [r10, r11, r12],
    [r20, r21, r22]])
    return rot_matrix
```

Explanation

We start by extracting each quaternion value to a variable with CoppeliaSim's quaternion order in mind, which is `[qx, qy, qz, qw]`.
Then, as each cell in the rotation matrix has its own formula, we solve each cell with the provided quaternion values.
Those solutions are then assigned to variables according to their cell location in the rotation matrix.
Lastly, we take all of those variables and combine them into a 3x3 matrix and return it.

*A comprehensive derivation for the formulas used can be found here:
https://www.songho.ca/opengl/gl_quaternion.html
(Be aware that the final result differs slightly. While $2(qw^2 + qx^2) - 1 = 1 - 2(qw^2 + qx^2)$ holds true theoretically, in code the latter form causes issues; most likely due to rounding errors (lack of precision in floats).)

Implementation 2)

```
def euler_from_matrix(R):
    eps = 5e-6

    ax, ay, az = 0.0 , 0.0 , 0.0

    r02 = R[0, 2]

    # case 1: r02 != |1|
    if 1 - r02 > eps and 1 + r02 > eps:
        ax = math.atan2(-R[1, 2], R[2, 2])
        ay = math.atan2(R[0, 2], math.sqrt(pow(R[0, 0], 2)+pow(R[0,
             1], 2)))
        az = math.atan2(-R[0, 1], R[0, 0])

    # case 2: r02 == +1
    elif 1 - r02 <= eps:
        ax = 0
        ay = math.pi/2
        az = math.atan2(R[1, 0], R[1, 1])

    # case 3: r02 == -1
    elif 1 + r02 <= eps:
        ax = 0
        ay = -math.pi/2
        az = math.atan2(R[1, 0], R[1, 1])

    return [ax, ay, az]
```

Explanation

For this implementation, we have to first check whether this rotation has experienced gimbal lock or not. To do so, we can check if $\sin(\beta)$ is between -1 and +1 (no gimbal lock) or if is ± 1 (gimbal lock).

In case 1 each angle is independent and has its own value, which can be calculated from the rotation matrix.

$\alpha = tan^{-1}(-r_{12}/r_{22})$
$\beta = tan^{-1}(-r_{02}/\sqrt{r_{00}^2 + r_{01}^2})$
$\gamma = tan^{-1}(-r_{01}/r_{00})$

In case 2 and 3 both the x-axis and z-axis are aligned due to β having reached $\pm 90^{\circ}$, meaning that we can only calculate their difference. Due to convention we set α to zero and solve for γ.

$\alpha = 0$
$\beta = \pm\pi/2$
$\gamma = tan^{-1}(r_{10}/r_{11})$

After calculating the euler angles, we return them.

Implementation 3)

```
def matrix_from_euler(angles):
    # euler angles
    alpha = angles[0]
    beta = angles[1]
    gamma = angles[2]

    # First row of the rotation matrix
    r00 = math.cos(beta)*math.cos(gamma)
    r01 = -(math.cos(beta)*math.sin(gamma))
    r02 = math.sin(beta)

    # Second row of the rotation matrix
    r10 = math.sin(alpha)*math.sin(beta)*math.cos(gamma) + math.cos
        (alpha)*math.sin(gamma)
    r11 = -math.sin(alpha)*math.sin(beta)*math.sin(gamma) + math.
        cos(alpha)*math.cos(gamma)
    r12 = -(math.sin(alpha)*math.cos(beta))

    # Third row of the rotation matrix
    r20 = -math.cos(alpha)*math.sin(beta)*math.cos(gamma) + math.
        sin(alpha)*math.sin(gamma)
    r21 = math.cos(alpha)*math.sin(beta)*math.sin(gamma) + math.sin
        (alpha)*math.cos(gamma)
    r22 = math.cos(alpha)*math.cos(beta)

    # 3x3 rotation matrix
    rot_matrix = np.array([[r00, r01, r02],
                           [r10, r11, r12],
                           [r20, r21, r22]])
    return rot_matrix
```

Explanation

Similarly to `convert_quaternion_to_matrix` we begin by extracting the given values from the parameter.
In this case it's the three euler angles α, β and γ.

After that we follow the matrix from the **Prep** section and assign each row the formulas as shown in **Step 2**. They then get calculated using the angles we extracted earlier and at last we combine the cells into a rotation matrix and return it.

Output on the next page

```
quat:  [0.0, 0.0, 0.0, 1.0]
[[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]
alpha: -0.0
beta : 0.0
gamma: -0.0
--------------------------------------------
[[ 1.  0.  0.]
 [-0.  1.  0.]
 [ 0. -0.  1.]]
```

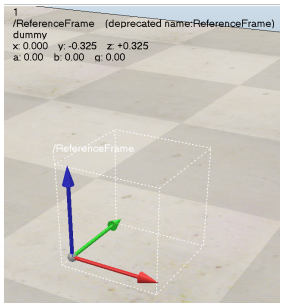


Figure 1: No rotation

```
quat:  [0.18214796572990696, 0.36758011983240724, -0.07843933778459997, 0.9892553402529751]
[[ 0.71984631  0.26200263  0.64278761]
 [ 0.00581325  0.92372884 -0.38302222]
 [-0.69410914  0.27945382  0.66341395]]
alpha: 30.000000000001968
beta : 40.000000000002636
gamma: -20.000000000001318
--------------------------------------------
[[ 0.71984631  0.26200263  0.64278761]
 [ 0.00581325  0.92372884 -0.38302222]
 [-0.69410914  0.27945382  0.66341395]]
```

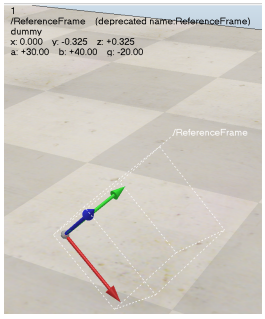


Figure 2: Rotation with alpha = 30° , beta = 40° and gamma = -20°

```
quat:  [0.1830127018922216, 0.6830127018922515, 0.1830127018922056, 0.6830127018921809]
[[-1.83361764e-13 -5.55111512e-17  1.00000000e+00]
 [ 5.00000000e-01  8.66025404e-01  5.17641485e-14]
 [-8.66025404e-01  5.00000000e-01 -8.94839758e-14]]
alpha: 0.0
beta : 90.0
gamma: 30.000000000001968
--------------------------------------------
[[ 5.30287619e-17 -3.86161780e-17  1.00000000e+00]
 [ 5.00000000e-01  8.66025404e-01 -0.00000000e+00]
 [-8.66025404e-01  5.00000000e-01  6.12323400e-17]]
```

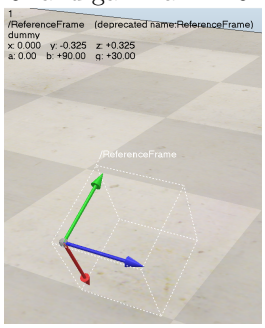


Figure 3: Rotation with alpha = 30°, beta = 90° and gamma = 0°
⇒ alpha = 0°, beta = 90° and gamma = 30°

```
quat:  [0.1830127018922216, -0.6830127018922515, -0.1830127018922056, 0.6830127018921809]
[[-1.03361764e-13  5.55111512e-17 -1.00000000e+00]
 [-5.00000000e-01  8.66025404e-01  5.17641485e-14]
 [ 8.66025404e-01  5.00000000e-01 -8.94839758e-14]]
alpha: 0.0
beta : -90.0
gamma: -30.000000000001968
--------------------------------------------
[[ 5.30287619e-17  3.86161780e-17 -1.00000000e+00]
 [-5.00000000e-01  8.66025404e-01 -0.00000000e+00]
 [ 8.66025404e-01  5.00000000e-01  6.12323400e-17]]
```

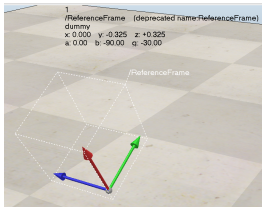


Figure 4: Rotation with alpha = 30°, beta = -90° and gamma = 0
⇒ alpha = 0°, beta = -90° and gamma = -30°

Task 2a)

Build a “snake” type robot in a 2D plane consisting of 6 rotational joints, rotating around the z-axis of the world coordinate system (WCS).

To connect the joints by links, you can use a primitive shape like a cuboid in size 0.25, 0.05, 0,05 in x, y and z. Ensure, that these cuboids are static (untick dynamic property). Also ensure that the joints are in passive mode. The snake robot in its zero position (all joints are zero) should look as in figure 1.

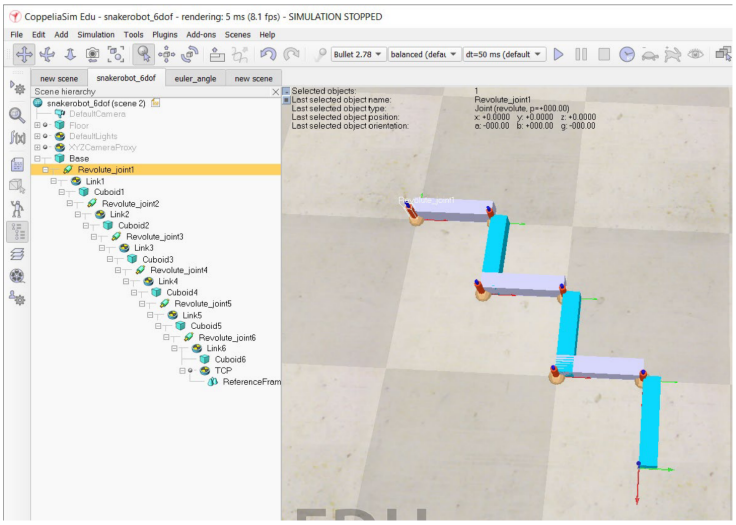
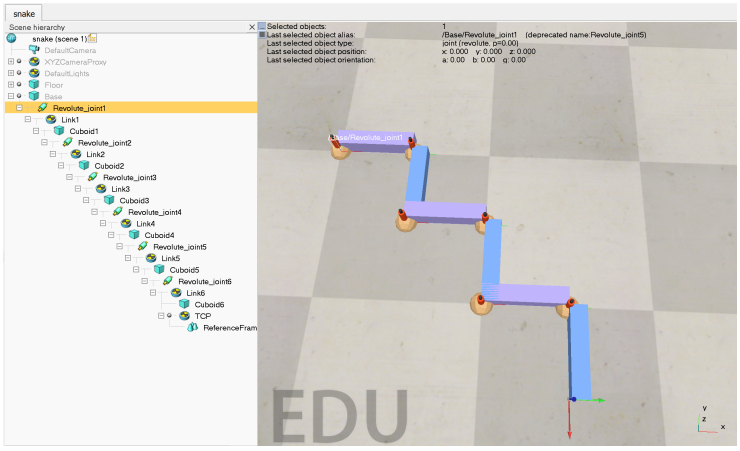


Figure 1: snake robot with 6 rotational joints in its zero position

Snake robot model



Made in CoppeliaSim 4.9.0 (rev. 6)

Prep

Translation and rotation for joints

Joint	Joint DOF	Tx	Ty	Tz	Rx	Ry	Rz	Remark
	-	0	0	0	0	0	0	from world to robot base system
1	Rz(q1)	0,25	0	0	0	0	-90°	from 1. to 2. joint
2	Rz(q1)	0,25	0	0	0	0	90°	from 2. to 3. joint
3	Rz(q1)	0,25	0	0	0	0	-90°	from 3. to 4. joint
4	Rz(q1)	0,25	0	0	0	0	90°	from 4. to 5. joint
5	Rz(q1)	0,25	0	0	0	0	-90°	from 5. to 6. joint
6	Rz(q1)	0,25	0	0	0	0	0	from 6. joint to TCP system

Task 2b)

b) Extend the python script `P2_Task2.py` to build the forward kinematic chain for the snake robot starting from the WCS to the tip (TCP coordinate system).

Do not hardcode the forward kinematic chain with respect to only this snake robot. Implement a generic solution, which would also work for other types of serial kinematic chains, see also the preparation of task 2.

Results for b)

1) The kinematic structure of the snake robot

```
snake_robot = np.array([
    [0, 0, 0, 0, 0, 0],             # WCS to 1. joint
    [0.25, 0, 0, 0, 0, -1.5708],    # 1. to 2.
    [0.25, 0, 0, 0, 0, 1.5708],     # 2. to 3.
    [0.25, 0, 0, 0, 0, -1.5708],    # 3. to 4.
    [0.25, 0, 0, 0, 0, 1.5708],     # 4. to 5.
    [0.25, 0, 0, 0, 0, -1.5708],    # 5. to 6.
    [0.25, 0, 0, 0, 0, 0]])         # 6. to TCP
```

2) Forward kinematics function

```
def forward_kinematics(rob, joints):
    TCP = t.identity_matrix()

    number_joints = len(joints)
    for i in range(1, number_joints+1):
        # Rotation(joint) * Translation * Rotation
        Rq = t.rotation_matrix(joints[i-1], [0, 0, 1])
        T = t.translation_matrix(rob[i][:3])
        R = t.euler_matrix(rob[i][3], rob[i][4], rob[i][5], 'rxyz')
        TCP = t.concatenate_matrices(TCP, Rq, T, R)
    return TCP
```

Explanation

The parameters.
`rob` is an array which holds the information regarding the robot's kinematic structure (c.f. `snake_robot` above).
`joints` is an array that contains each joint's position (in radian).

The function's body.
At the start we initialize the TCP with the identity matrix. Then, for better readability, we save the length of the joints-array in the variable `number_joints`. In the for-loop we iterate one time more than the number of joints present in the robot. This is because the rob-array contains one more element.

The first element of the rob-array also gets skipped, which is why we start iterating at i = 1 and thus have to subtract 1 from the current index when accessing elements from the joints-array as it holds only 6 elements.

In the for-loop's body we calculate the TCP's transformation matrix.
First, we calculate `Rq`, which is a homogeneous rotation matrix about the Z-axis based on the joint's rotation. It represents the movement allowed by the joint (its DoF).
Next, we calculate `T`, which is a homogeneous translation matrix that accounts for the link translation. This describes the translation from the current joint to the next frame (e.g., the next joint or the TCP).
Finally, we calculate `R`, which is a rotation matrix that contains the fixed rotation of the link. It represents the constant orientation of the link relative to the previous frame.
At last, we multiply the current TCP, Rq, T and R from left to right.
Once the for-loop has ended we have the final pose of the TCP, which we return.

*All matrices are 4x4